



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
по дисциплине «Разработка баз данных»

Студент группы
ИКБО-50-23

ИКБО-50-23. Астахов С.П..

(подпись)

Преподаватель

Мажей Я.В

(подпись)

Москва 2025 г.

Содержание

1. Цель работы и постановка задач.....	3
2. Ход выполнения работы.....	5
2.1. Скриншоты самой главной таблицы с 20 тысячей записей.....	5
2.2. Задание 1.....	5
2.2.1. ыва.....	5
3. Вывод.....	6
4. Дополнительная литература.....	7

1. Цель работы и постановка задач

Цель: Целью данной практической работы является формирование у студентов практических навыков анализа и оптимизации производительности SQL-запросов, а также освоение механизмов управления транзакциями для обеспечения целостности данных (согласно принципам ACID) в СУБД PostgreSQL.

Постановка задачи:

Наполнить одну из ключевых таблиц вашей БД большим объемом данных (не менее 20 000 строк). Это обязательно для корректной демонстрации работы оптимизатора.

Задание №1: анализ и оптимизация (3 сценария)

Определить три различных «медленных» запроса к вашей БД, которые можно оптимизировать с помощью разных типов индексов (например, стандартный B-Tree, индекс по выражению, частичный/отфильтрованный индекс).

Если в вашей базе недостаточно данных, выполните для одной из своих таблиц действия по автоматической генерации содержимого, описанные в разделе «Подготовка базы данных».

Для каждого из 3-х сценариев:

1. Выполнить анализ запроса «КАК ЕСТЬ» (без индекса) с помощью EXPLAIN ANALYZE.
2. Привести план выполнения «ДО», письменно проанализировать его и выявить причину низкой производительности (например, Seq Scan).
3. Создать необходимый INDEX для оптимизации.
4. Повторно выполнить EXPLAIN ANALYZE.

5. Привести план выполнения «ПОСЛЕ», демонстрирующий использование индекса (например, Index Scan).
6. Обязательно сформировать сравнительную таблицу (см. Таблица 1), демонстрирующую разницу в производительности (план, Execution Time) «ДО» и «ПОСЛЕ».

Задание №2: демонстрация атомарной транзакции (COMMIT).

По примеру раздела 2.2 реализуйте в своей базе одну бизнес-операцию (минимум две связанные операции изменения данных) внутри транзакции BEGIN...COMMIT.

Задание №3: демонстрация отката транзакции (ROLLBACK).

Адаптировать приведённый в разделе 2.3 SQL-скрипт, моделирующий сбой операции, под свою предметную область, повторив описанные действия.

Задание №4: моделирование аномалии «Неповторяемое чтение».

Используя два редактора SQL, смоделировать проблему «неповторяемое чтение» на уровне изоляции по умолчанию (READ COMMITTED) по приведённому в разделе 2.4 образцу.

Задание №5: устранение аномалии «Неповторяемое чтение».

Повторить моделирование из Задания №4, но с использованием уровня изоляции REPEATABLE READ. В качестве образца использовать раздел 2.5.

После каждого задания задокументировать все шаги и результаты, сделать выводы.

2. Ход выполнения работы

2.1. Скриншоты самой главной таблицы с 20 тысячей записей

Вначале напишем скрипт для заполнения таблицы случайными значениями:

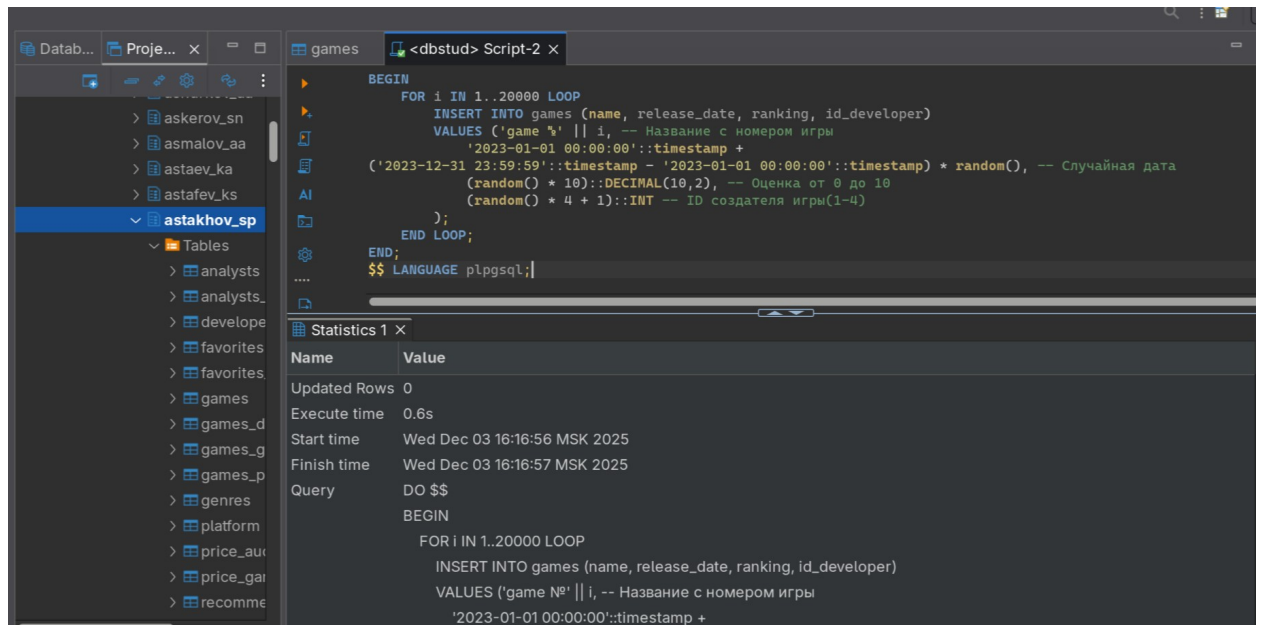


Рисунок 1 - Скрипт для заполнения таблицы

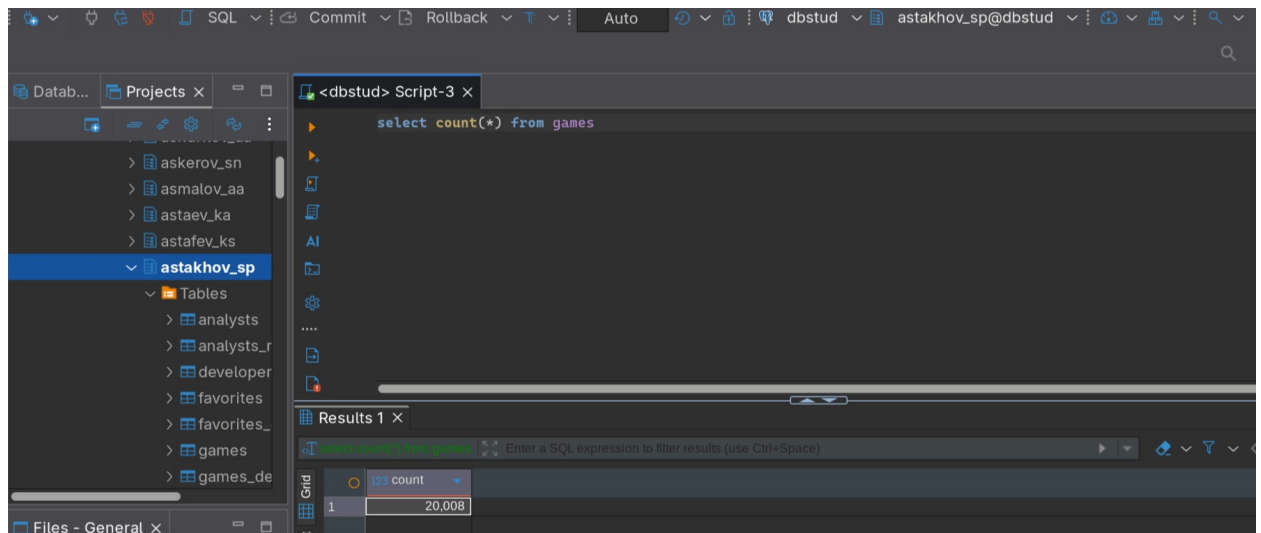


Рисунок 2 - Кол-во строк таблицы games

The screenshot shows a database client interface with a project named 'astakhov_sp'. The 'games' table is selected, and its data is displayed in a grid view. The table has 6 columns: id_game, AZ name, release_date, ranking, id_developer, and id_genres. The data includes 18 rows, with the last row being 'game №9'.

id_game	AZ name	release_date	ranking	id_developer	id_genres
1	Dota 2	2013-07-09	7.2	2	3
2	Half-Life 2	2004-11-16	9	2	8
3	The Talos Principle	2014-12-11	8.5	4	2 [+1]
4	Hollow Knight	2017-02-24	8.2	1	1 [+1]
5	Hollow Knight: Silksong	2025-09-04	9.1	1	1 [+1]
6	Counter-Strike 2	2023-09-27	6.1	2	10
7	Hearts of Iron IV	2016-06-06	8.4	3	3
8	Portal	2007-10-10	9.5	2	2 [+3]
9	game №1	2023-12-13	3.87	3	NUL
10	game №2	2023-10-31	8.95	4	NUL
11	game №3	2023-05-31	0.96	5	NUL
12	game №4	2023-08-13	1.58	3	NUL
13	game №5	2023-11-23	6.33	4	NUL
14	game №6	2023-09-20	9.78	4	NUL
15	game №7	2023-09-21	8.03	5	NUL
16	game №8	2023-12-03	6.5	3	NUL
17	game №9	2023-03-11	4.23	3	NUL

Рисунок 3 - Таблица games

2.2. Задание №1: анализ и оптимизация

2.2.1. Сценарий 1

Шаг 1. Анализ «ДО» (без индекса)

The screenshot shows a database client interface with a project named 'astakhov_sp'. The 'games' table is selected, and the query 'EXPLAIN ANALYZE SELECT * FROM games WHERE name = 'Hearts of Iron IV';' is executed. The results show the execution plan for the query.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Seq Scan on games	0.00	419.10	61	0.064	3.353	1
2	Filter: ((name)::text = 'Hearts of Iron IV')::text)						
3	Rows Removed by Filter: 20007						
4	Planning Time: 0.117 ms						
5	Execution Time: 3.411 ms						

Рисунок 4 - Результат запроса до индекса

Шаг 2. Создание индекса

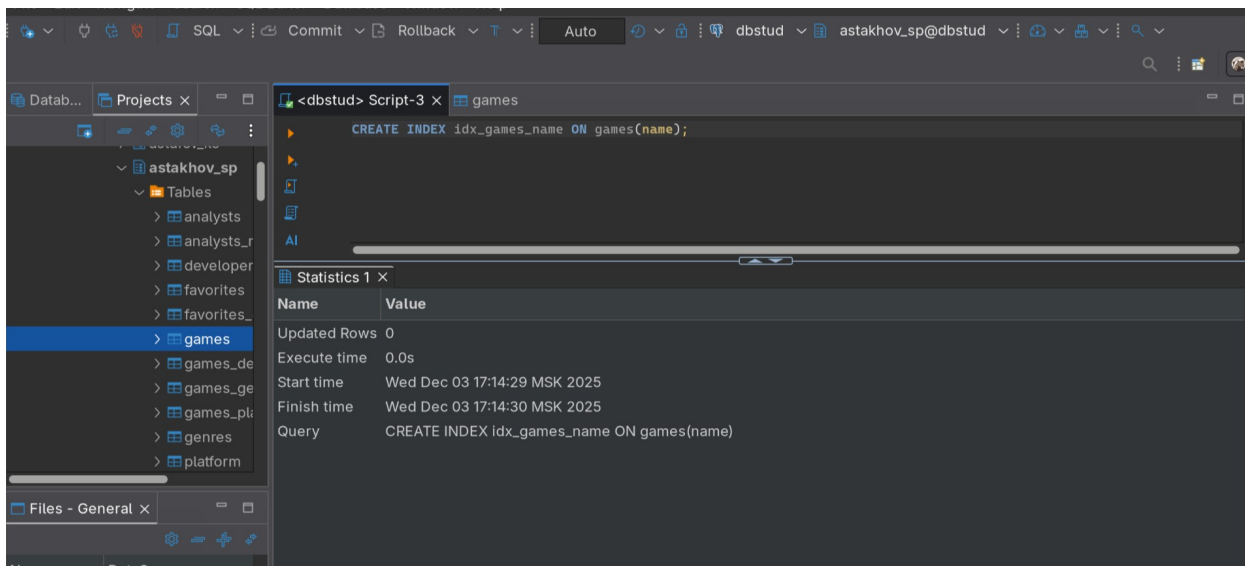


Рисунок 5 - Создаём B-Tree индекс

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

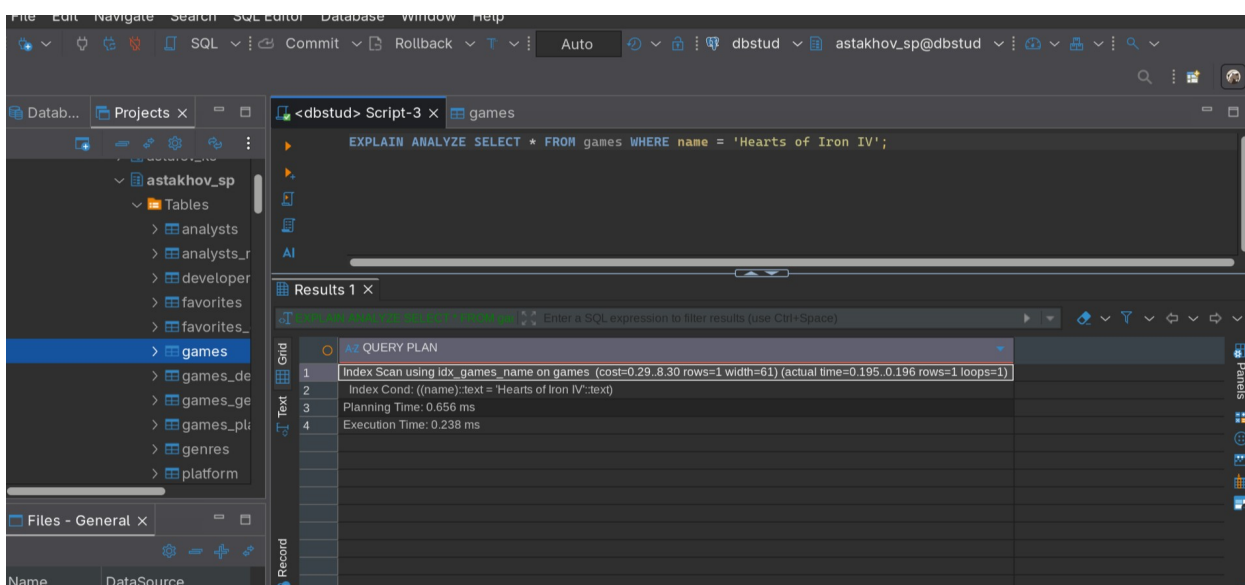


Рисунок 6 - Результат запроса после индекса

Шаг 4. Сравнительная таблица

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Index Scan	Выбор отличается
Execution Time	3.411 ms	0.238 ms	Запрос ускорился примерно в 14 раз

2.2.2. Сценарий 2

Шаг 1. Анализ «ДО» (без индекса)

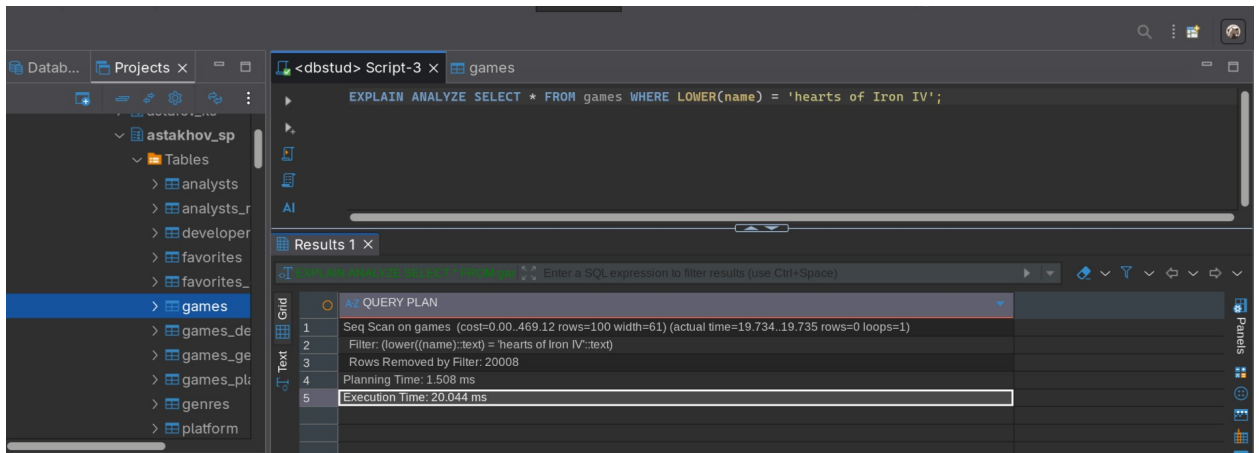


Рисунок 7 - Результат запроса до индекса

Шаг 2. Создание индекса

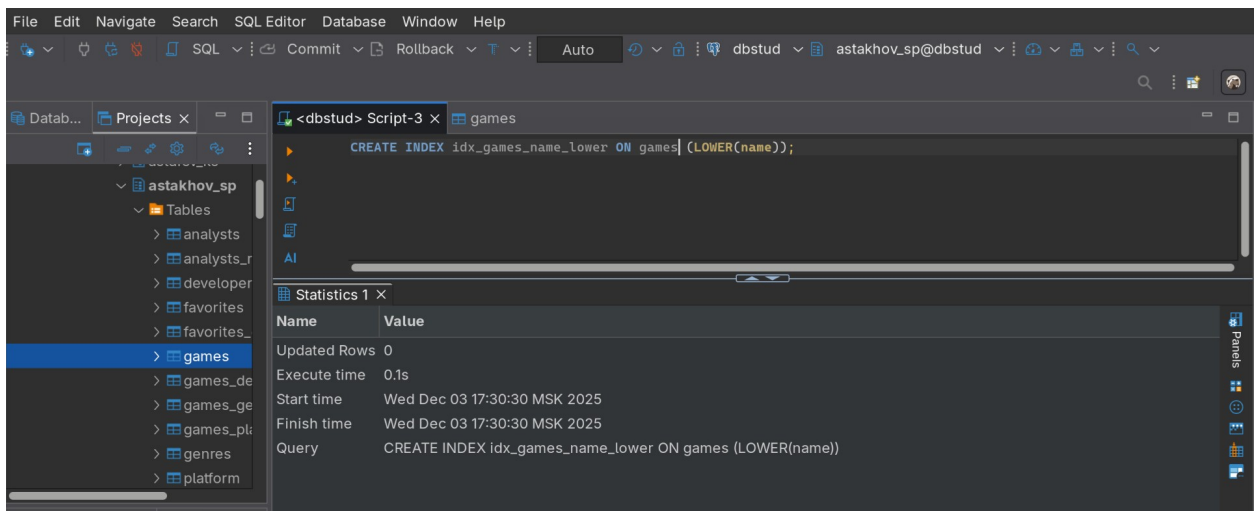


Рисунок 8 - Создаём индекс по выражению

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

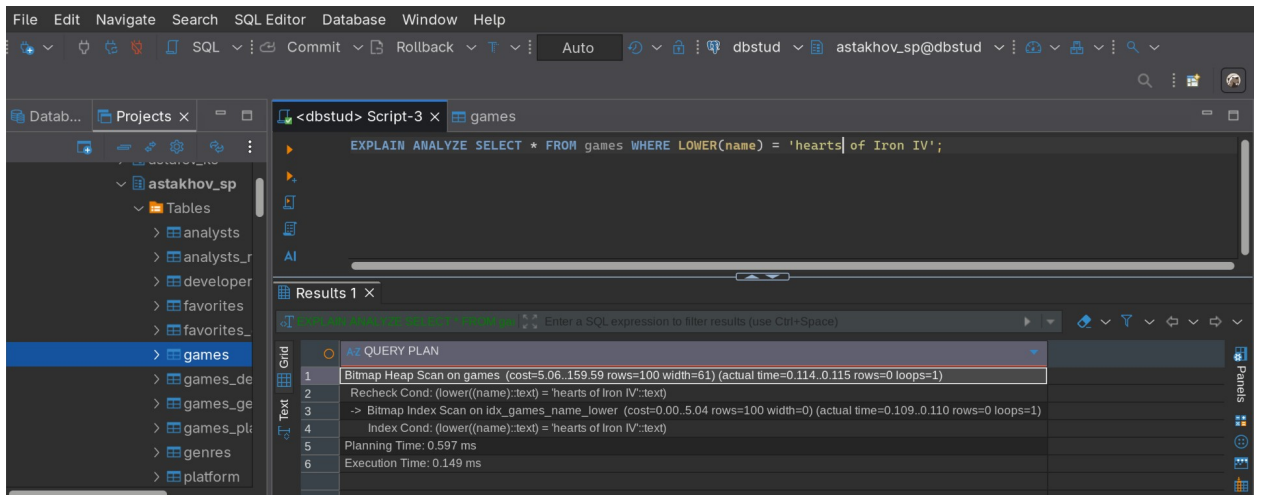


Рисунок 9 - Результат запроса после индекса

Шаг 4. Сравнительная таблица

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Bitmap Index Scan	Выбор отличается
Execution Time	20.044 ms	0.149 ms	Запрос ускорился примерно в 134 раз

2.2.3. Сценарий 3

Шаг 1. Анализ «ДО» (без индекса)

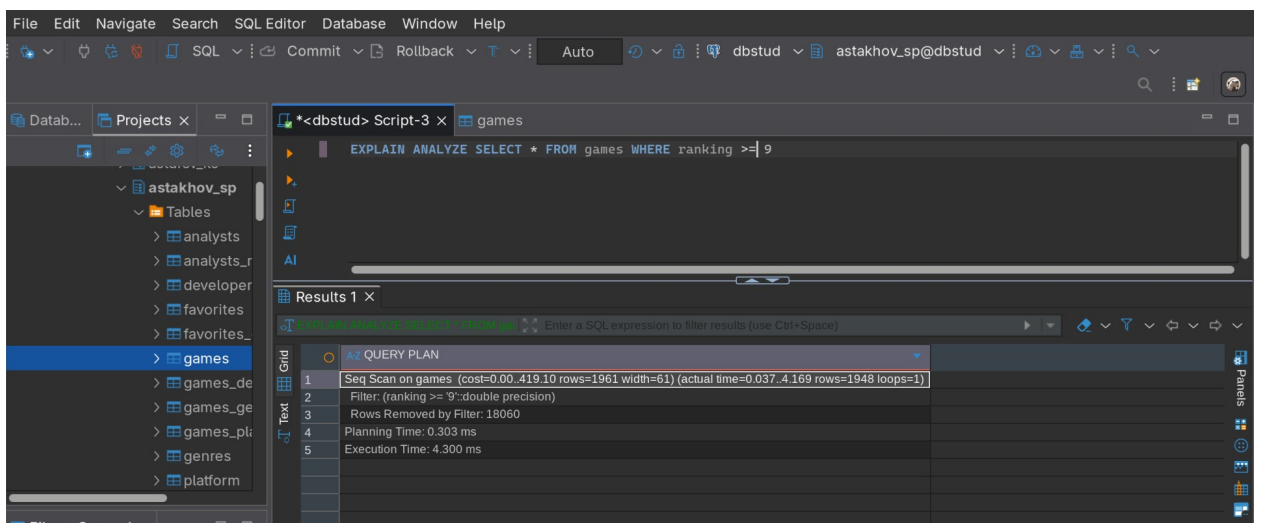


Рисунок 10 - Результат запроса до индекса

Шаг 2. Создание индекса

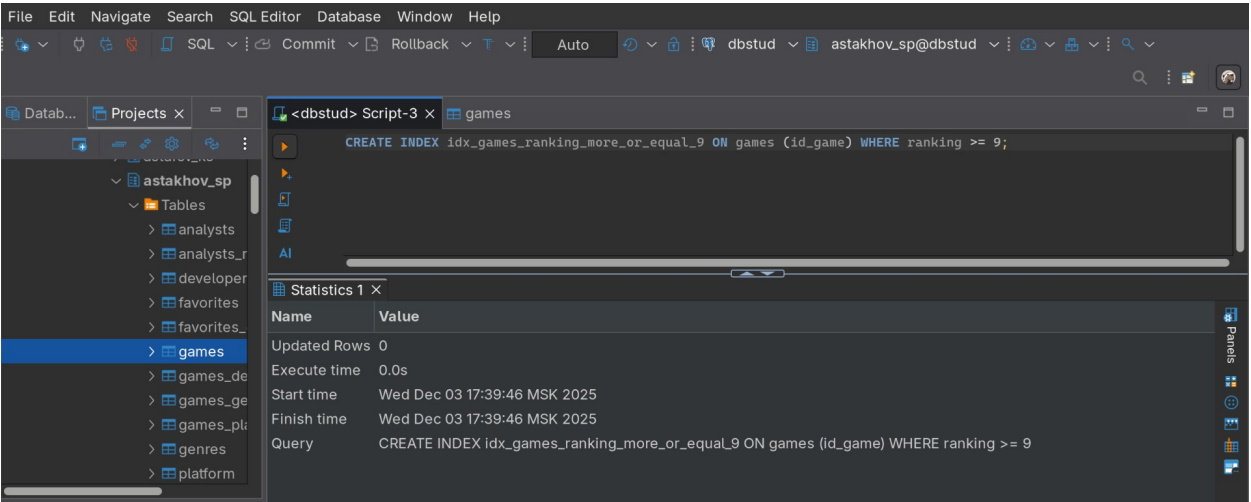


Рисунок 11 - Создаем частичный индекс

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

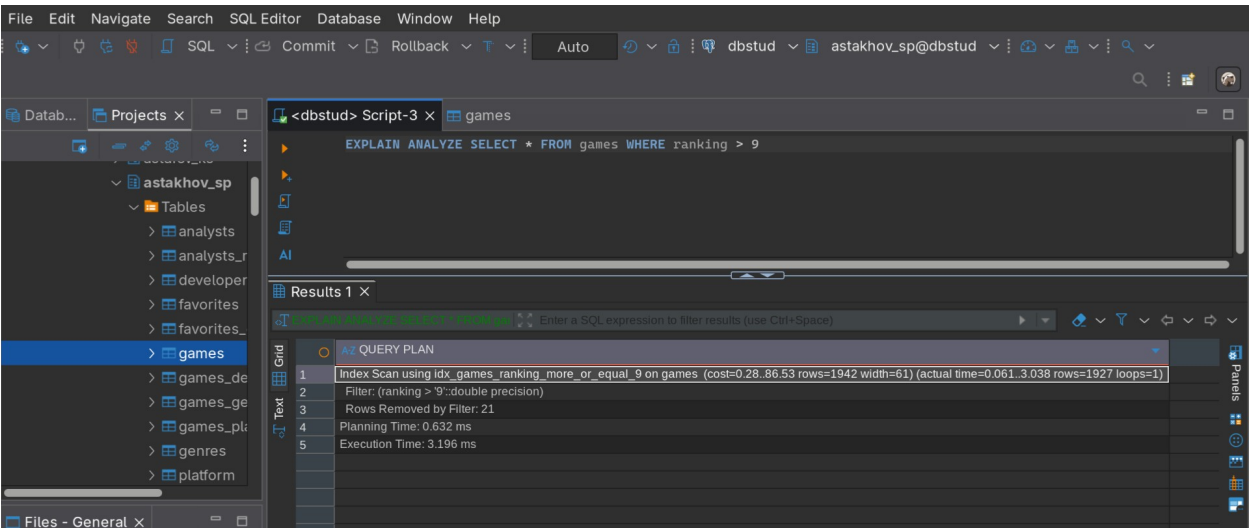


Рисунок 12 - Результат запроса после индекса

Шаг 4. Сравнительная таблица

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Index Scan	Выбор отличается
Execution Time	4.300 ms	3.196 ms	Запрос ускорился примерно в 1,3 раза

2.3. Задание №2: демонстрация атомарной транзакции (COMMIT).

Перед началом, добавим в таблицу user_ столбцы для работы с атомарными операциями:

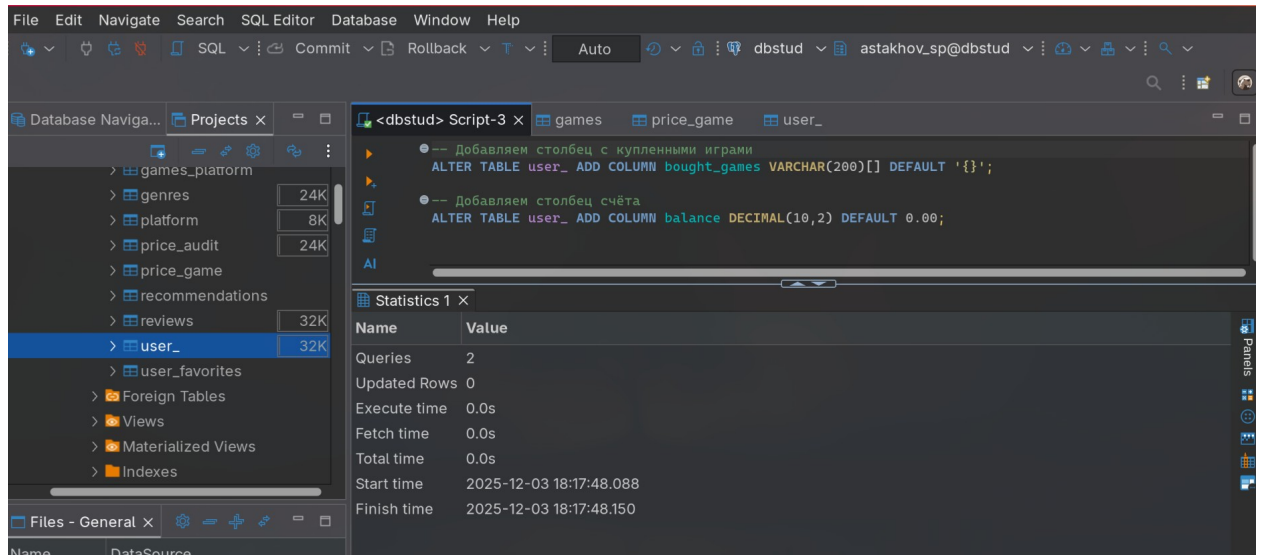


Рисунок 13 - Добавление новых столбцов

И заполним случайными значениями:

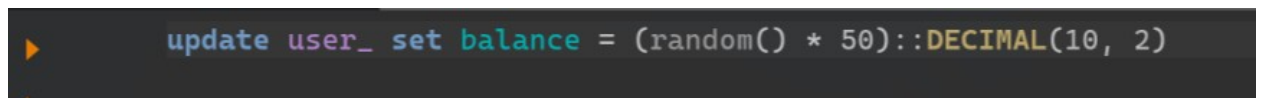


Рисунок 14 - Обновление данных счета

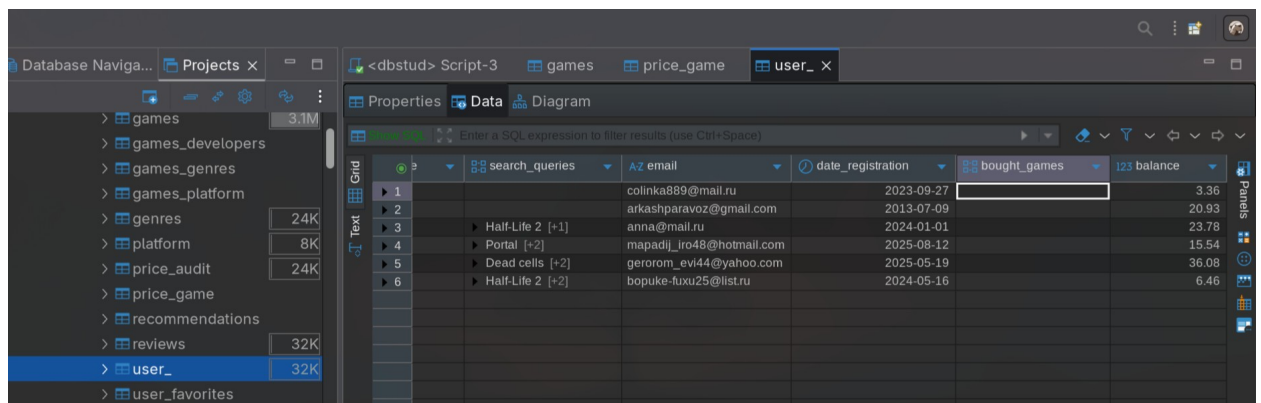


Рисунок 15 - Таблица user_ до атомарной операции

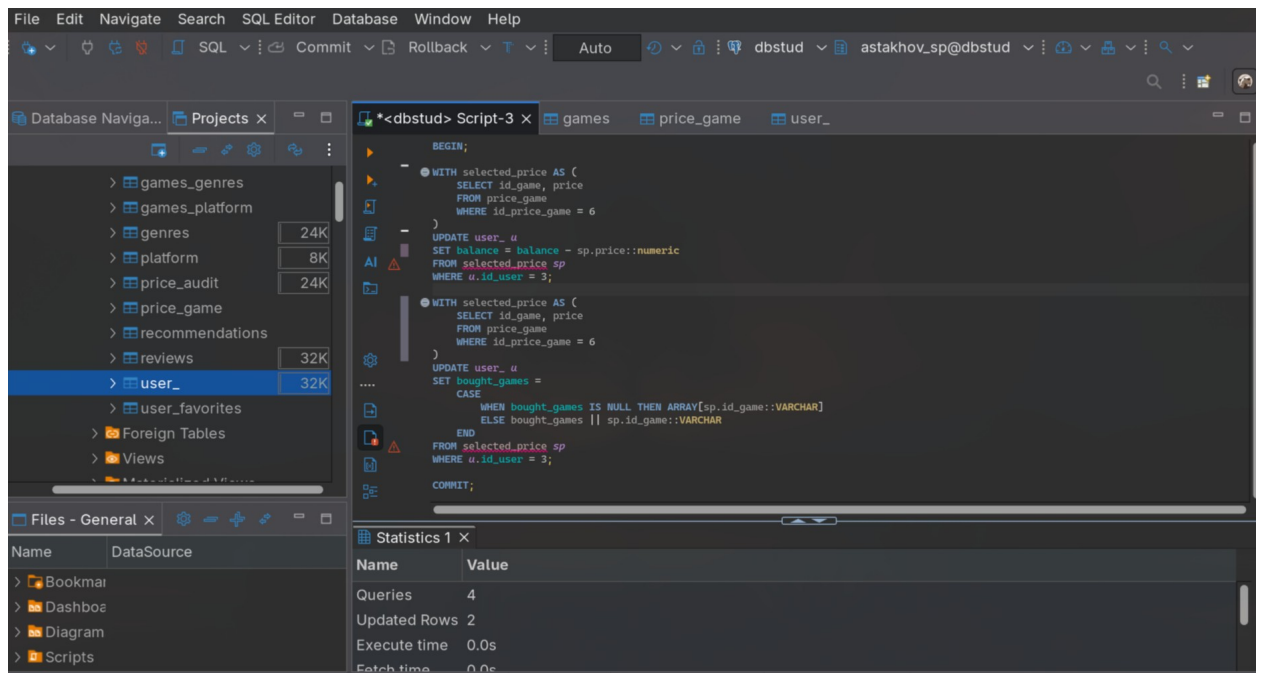


Рисунок 16 - Выполняем атомарную операцию

The screenshot shows the 'user_' table data in a grid view. The table has the following columns: 'search_queries', 'email', 'date_registration', 'bought_games', and 'balance'. The data is as follows:

	search_queries	email	date_registration	bought_games	balance
1		colinka889@mail.ru	2023-09-27		3.36
2		arkashparavoz@gmail.com	2013-07-09		20.93
3	Half-Life 2 [+1]	anna@mail.ru	2024-01-01		23.78
4	Portal [+2]	mapadji_iro48@hotmail.com	2025-08-12		15.54
5	Half-Life 2 [+2]	bopuke-fluxu25@list.ru	2024-05-16		6.46
6	Dead cells [+2]	gerorom_ev44@yahoo.com	2025-05-19	1	27.98
6.1				6	

Рисунок 17 - Таблица user_ после атомарной операции

2.4. Задание №3: демонстрация отката транзакции (ROLLBACK).

Добавим функции для вызова ошибки, чтоб сработал rollback:

```

-- Проверка на условие есть ли игра
DO $$
DECLARE
    game_id_val INTEGER;
BEGIN
    SELECT id_game INTO game_id_val
    FROM price_game
    WHERE id_price_game = 6;

    -- Если игра уже в массиве - ОШИБКА
    IF EXISTS (
        SELECT 1
        FROM user_
        WHERE id_user = 3
        AND game_id_val::VARCHAR = ANY(bought_games)
    ) THEN
        RAISE EXCEPTION 'Игра % уже куплена! Нарушение уникальности покупки.', game_id_val;
    END IF;
END $$;

```

Рисунок 18 - Код для вызова ошибки

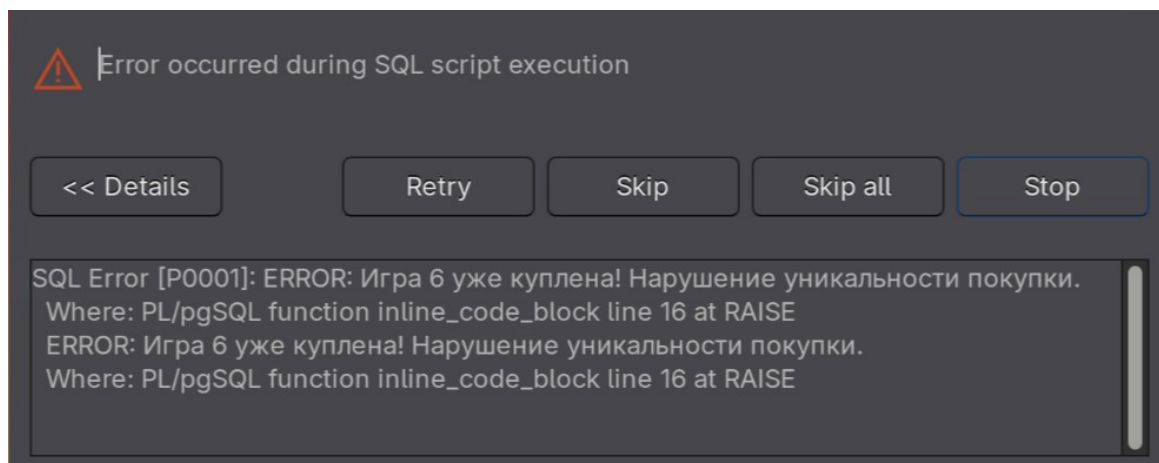


Рисунок 19 - Ошибка

	search_queries	A-Z email	date_registration	bought_games	balance
1	Dead cells [+2]	gerorom_evi44@yahoo.com	2025-05-19	6	27.98
2		colinka889@mail.ru	2023-09-27		3.36
3		arkashparavoz@gmail.com	2013-07-09		20.93
4	Half-Life 2 [+1]	anna@mail.ru	2024-01-01		23.78
5	Portal [+2]	mapadij_iro48@hotmail.com	2025-08-12		15.54
6	Half-Life 2 [+2]	bopuke-fuxu25@list.ru	2024-05-16		6.46

Рисунок 20 - Таблица после rollback

2.5. Задание №4: моделирование аномалии «Неповторяемое чтение».

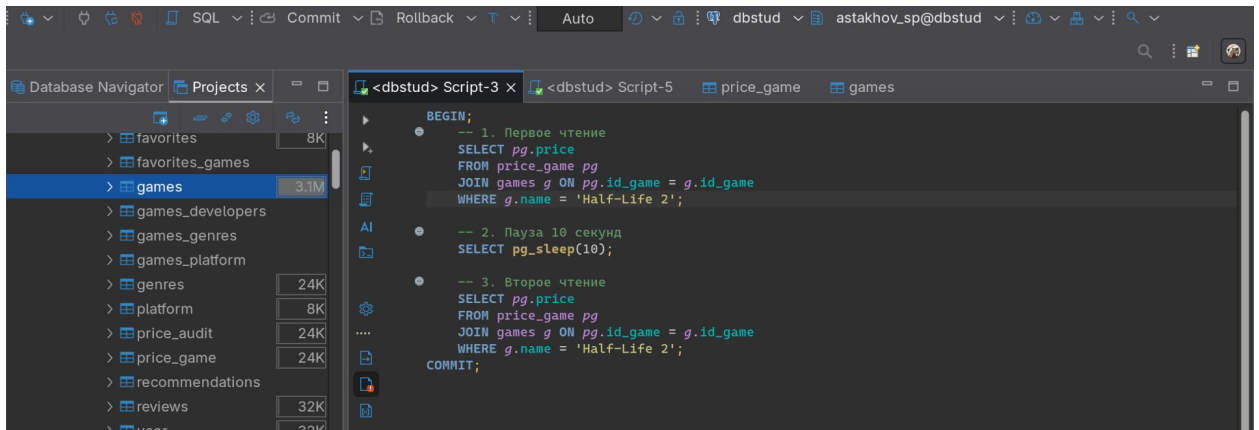


Рисунок 21 - Скрипт А

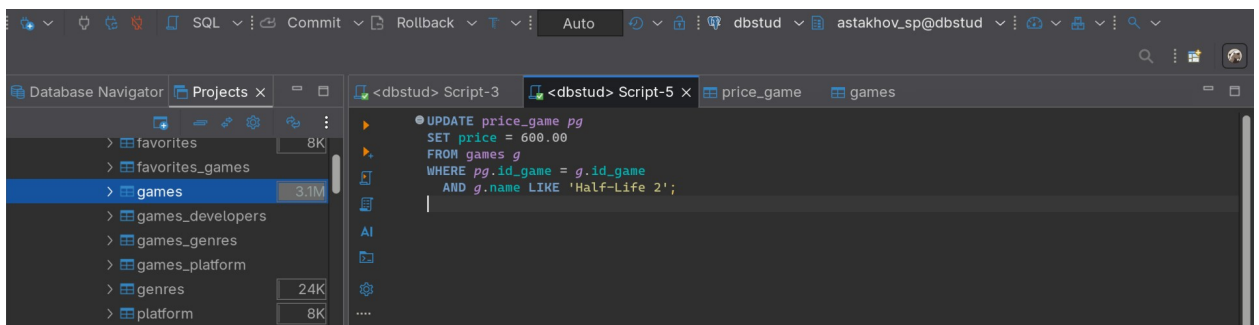


Рисунок 22 - Скрипт В

The screenshot shows a table view of the 'price_game' table. The table has two columns: 'id' and 'price'. The first row shows '1' and '8.1'. The second row shows '123' and 'price'.

	id	price
Grid	1	8.1
Text	123	price

Рисунок 23 - Цена при первом чтении

	id	price
1	1	600
2	123	price

Рисунок 24 - Цена при повторном чтении, спустя 10 секунд

2.6. Задание №5: устранение аномалии «Неповторяемое чтение».

Откатим цену на ту, которая была до этого:

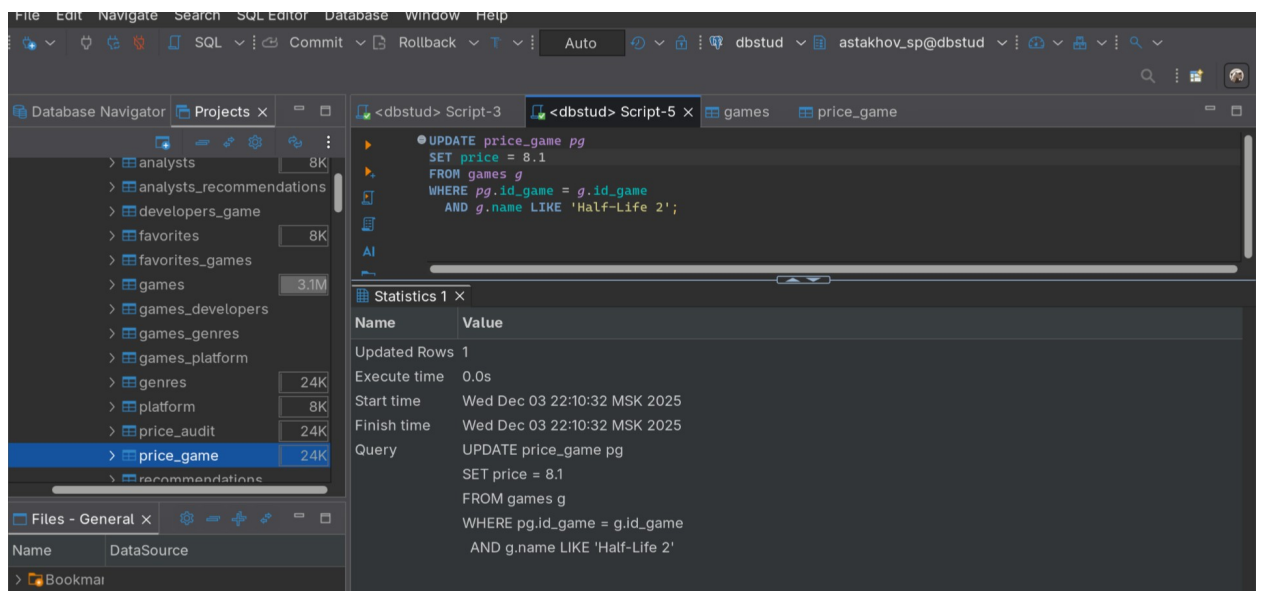


Рисунок 25 - Обновление прошлой цены игры

Добавим строчку с указанием изоляции:

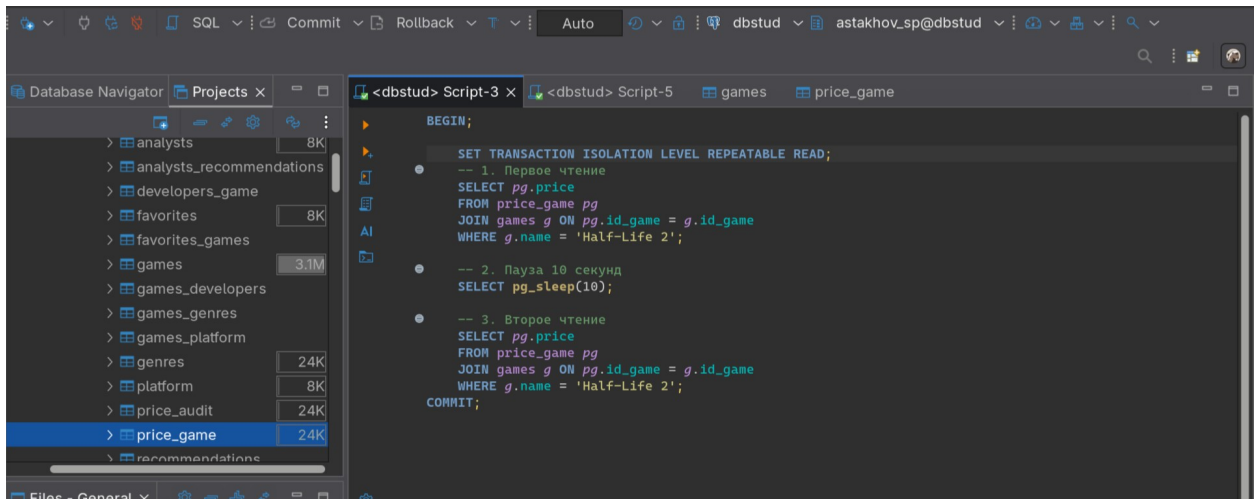


Рисунок 26 - Указание REPEATABLE READ

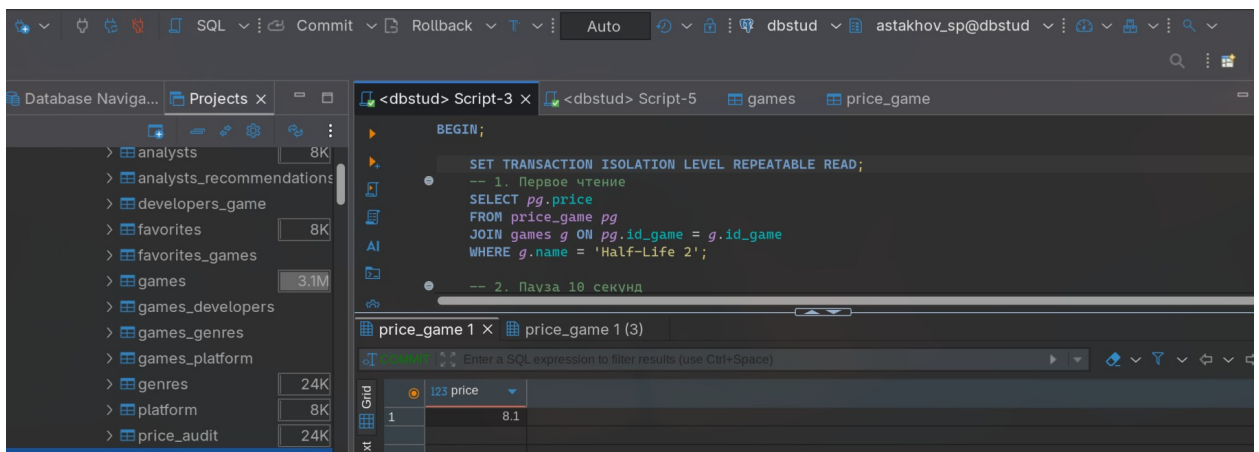


Рисунок 27 - Цена при первом чтении

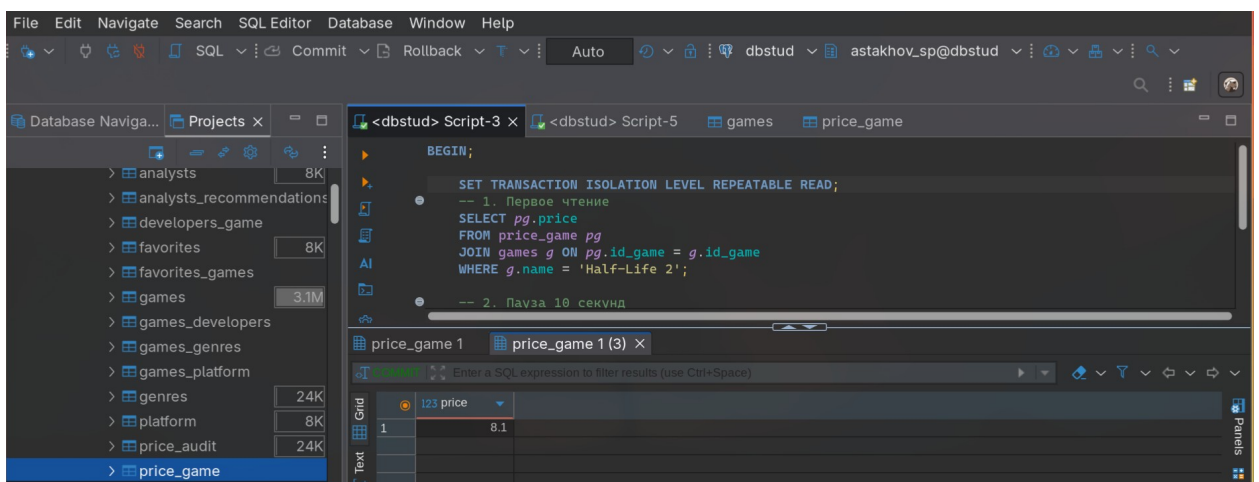


Рисунок 28 - Цена при повторном чтении, спустя 10 секунд

3. Вывод

В данной практической работе мы изучили разные виды индексов, выполнили атомарные операции и выходы из них, а также аномалии.

4. Дополнительная литература

1. Литература_Разработка_баз_данных_Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Гиперссылка
2. Литература_Разработка_баз_данных_Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил.
3. Путеводитель по базам данных. — М.: ДМК-Пресс, 2024. — 520 с. ISBN 978-5-93700-287-7